

SYNERGETICS • INSTRUCTOR-LED WORKSHOP

RAG Foundations & Pipeline

Foundations, Pipeline & Applied Exercise

Grounding LLMs in enterprise data • Azure OpenAI & Azure AI Search • Hands-on RAG pipeline

Learning Objectives

1 Understand RAG

Grasp how Retrieval-Augmented Generation grounds LLM outputs in your own enterprise data.

2 Design a RAG pipeline

Walk through ingestion, chunking, embeddings, vector search, and generation end to end.

3 Build a working RAG app

Implement ingestion, indexing, retrieval, and grounded generation yourself in the Applied Exercise.

4 Use Azure-native tooling

Work with Azure OpenAI and Azure AI Search to power every stage of the pipeline.

Prerequisites & Assumed Knowledge

1

GenAI & LLM Fundamentals

Comfortable with concepts like prompts, tokens, and what a large language model is.

2

Basic Programming

Python is recommended — we'll read and adapt small code snippets, not write from scratch.

3

Enterprise Data & APIs

Familiarity with how your organization's data is stored and exposed (documents, databases, APIs).

4

Microsoft Azure

Working knowledge of Azure is recommended; we'll be hands-on in Azure OpenAI and Azure AI Search.

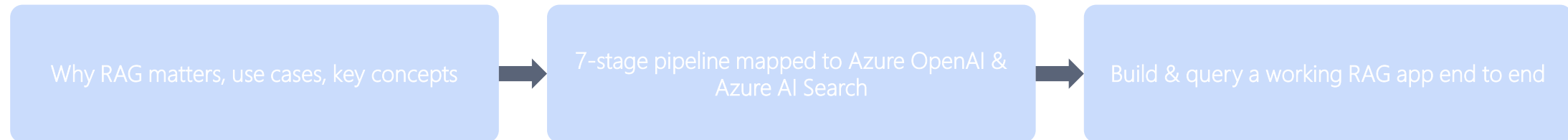
Session Overview

This session is entirely focused on Retrieval-Augmented Generation — from why RAG matters, to how a complete RAG pipeline is assembled, to building and querying a working RAG application yourself.

Session flow



What each phase covers



Quick Recap: Generative AI & LLM Fundamentals

CONCEPT

- Large Language Models (LLMs) predict the next token based on patterns learned from massive training data.
- They generate fluent, context-aware text — but everything they 'know' is frozen at training time.
- A prompt plus the model's context window is the only information the model has access to at inference time.

Two hard limits to remember

Knowledge cutoff

The model has no awareness of anything after its training data ended.

Context window

It only 'sees' what's in the current prompt — it cannot query your databases or file shares on its own.

RAG vs. a Standalone LLM

CONCEPT

Without Grounding

- Answers only from frozen training data
- Confidently states incorrect facts (hallucination)
- No visibility into your internal policies, tickets, or contracts
- No way to cite a source for its answer

With Grounding (RAG)

- Answers pulled from your live, current enterprise data
- Responses anchored to retrieved evidence, not guesswork
- Can reach policies, tickets, contracts, wikis — anything indexed
- Can show the source passages behind an answer

What Is Retrieval-Augmented Generation?

CONCEPT

RAG is a pattern that retrieves relevant information from an external knowledge source and inserts it into the model's prompt before generation — so the model answers using evidence, not memory alone.

Why it matters

Cuts hallucination, keeps answers current, avoids costly retraining or fine-tuning.

How it works (in brief)

Retrieve relevant passages → insert into the prompt → the model generates a grounded answer.

Key idea

Separates 'what the model knows' from 'what the model is told' at query time.

RAG in one line

*"Don't ask the model to remember.
Ask it to read, then answer."*

Retrieval + Generation = RAG

No model weights change. You are changing what the model sees, not what it is.

RAG vs. Fine-Tuning

Dimension	RAG	Fine-Tuning
Best for	Answering questions from data that changes often — policies, tickets, live docs	Teaching the model a new skill, style, or format it doesn't know at all
Keeping data current	Update the index any time — the model has fresh answers instantly	Requires retraining whenever the underlying knowledge changes
Cost & effort	Low — index your documents and start querying within hours	High — needs curated training data, compute, and ML expertise

RAG in Practice: Enterprise Use Cases

EXAMPLES

1 Internal Knowledge Assistant

Employees ask natural-language questions against HR policy, IT, or engineering wikis.

2 Customer Support Copilot

Agents get grounded answers from product manuals and past ticket resolutions.

3 Contract & Policy Analysis

Legal/compliance teams query large contract sets and get answers with clause citations.

4 Enterprise Search

Search across SharePoint, file shares, and databases returns synthesized answers, not just links.

Key Components of a Basic RAG Architecture

CONCEPT

A basic RAG system is assembled from a small set of building blocks that work together to ground the model's answers in your own data.

Why these pieces matter

- Each piece has one job — swap or upgrade any one without redesigning the whole system
- The same components appear in nearly every production RAG system, just with different vendors underneath
- Knowing the pieces individually makes debugging a bad answer far easier — you know exactly where to look

Core building blocks

Embedding Model

Converts text — documents and queries — into numeric vectors that capture meaning

Vector Store

Stores chunks + embeddings; Azure AI Search is our vector store today

Retriever

Finds the top-K most relevant chunks for a query via similarity search

Generator

Produces the final answer, grounded in the retrieved context

RAG Architecture: Ingestion & Chunking

CONCEPT

What & why

Ingestion pulls raw content (PDFs, wikis, tickets, databases) into the pipeline. Chunking splits that content into smaller passages so retrieval can return precise, relevant pieces instead of entire documents.

Common chunking strategies

- **Fixed-size** — Split every N tokens, with overlap to preserve context across boundaries.
- **Semantic / structural** — Split on natural boundaries — headings, paragraphs, sections.
- **Recursive** — Try larger units first, fall back to smaller splits only where needed.

Get chunk size wrong and...

Too small:

Fragments lose context; retrieval returns disconnected sentences.

Too large:

Irrelevant text dilutes the retrieved passage and wastes context window.

Rule of thumb: start around 300–500 tokens with ~10-15% overlap, then tune against real queries.

RAG Architecture: Embeddings & Vector Stores

CONCEPT

An embedding model converts text into a numeric vector that captures its meaning. Passages with similar meaning end up close together in vector space — which is what makes semantic search possible.

Vector store options

- Azure AI Search — managed, hybrid (keyword + vector), enterprise security built in
- Open-source (e.g., FAISS, pgvector) — flexible, self-managed, good for prototyping
- Specialized vector DBs — purpose-built for large-scale similarity search

Generating an embedding (Azure OpenAI)

```
from openai import AzureOpenAI

client = AzureOpenAI(
    azure_endpoint=ENDPOINT,
    api_key=API_KEY,
    api_version="2024-06-01"
)

response = client.embeddings.create(
    model="text-embedding-3-large",
    input=chunk_text
)

vector = response.data[0].embedding
# store the vector alongside chunk_text
# in Azure AI Search index
```

RAG Architecture: Retrieval & Generation

CONCEPT

1. Embed the query

The user's question is converted into a vector using the same embedding model.

2. Similarity search

The vector store returns the top-k most similar chunks (often combined with keyword filters).

3. Augment the prompt

Retrieved chunks are inserted into the prompt alongside the user's original question.

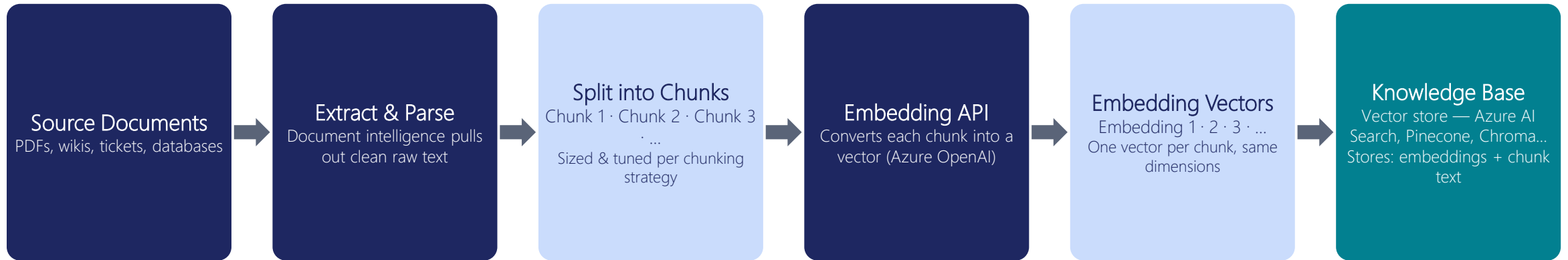
4. Generate

The LLM produces an answer grounded in the retrieved evidence — ideally with citations.

RAG Architecture: Ingestion Pipeline — Deep Dive

OFFLINE · INDEXING TIME

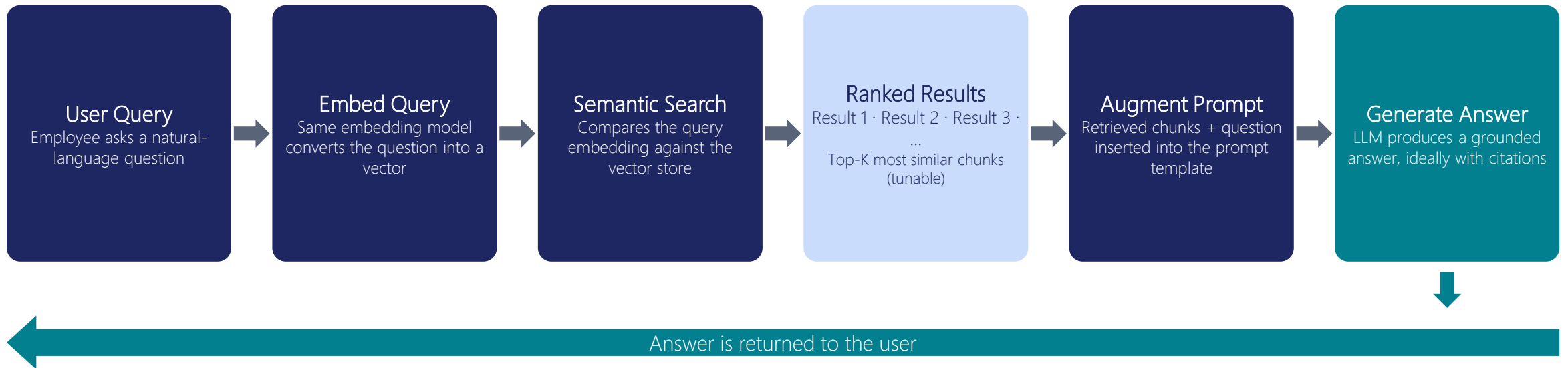
Full pipeline recall: Ingest → Chunk → Embed → Store (offline, this slide) → Retrieve → Augment → Generate (online, next slide).



This pipeline runs once, or on a schedule, whenever your source documents are added or updated — building the index the retrieval pipeline queries.

RAG Architecture: Retrieval Pipeline — Deep Dive

ONLINE · QUERY TIME



This sequence runs live on every user query. Steps 1–6 typically complete in under a second or two.

Demonstration: Retrieval + Generation

DEMONSTRATION

```
# 1. Embed the incoming query
query_vector = embed(user_question)

# 2. Search the vector index (Azure AI Search)
results = search_client.search(
    search_text=user_question,
    vector_queries=[{
        "vector": query_vector,
        "k_nearest_neighbors": 5,
        "fields": "content_vector"
    }]
)
context = "\n\n".join([r["content"] for r in results])

# 3. Augment the prompt
prompt = f"""Answer using ONLY the context below.
Context:
{context}

Question: {user_question}"""

# 4. Generate a grounded response
answer = chat_client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": prompt}]
)
```

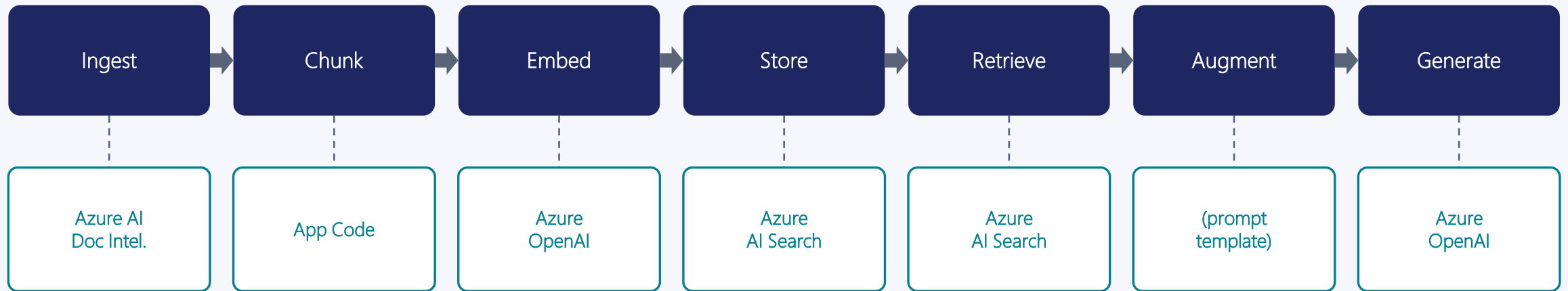
What to watch for

- Same embedding model for queries and documents
- "Answer using ONLY the context" reduces hallucination
- `k_nearest_neighbors` is a tunable retrieval parameter
- This is the minimum viable RAG loop — everything else we build today extends exactly this

Azure-Native RAG Tooling

Service	Role in the Pipeline	Key Capability
Azure OpenAI Service	Generates embeddings and the final grounded answer	Enterprise-grade GPT and embedding models with SLAs and regional data residency
Azure AI Search	Stores and searches vectors + keywords (the vector store)	Native hybrid search: vector + keyword + semantic re-ranking in one query
Azure AI Document Intelligence	Extracts clean text and structure from PDFs, scans, and other source documents during ingestion	Prebuilt and custom models for layout, tables, and forms — feeds directly into chunking

Mapping Azure Services to the Pipeline



One service can appear more than once — Azure OpenAI both embeds and generates; Azure AI Search both stores and retrieves.

Best Practices: Building Reliable RAG

1 Tune chunk size against real queries

Don't guess — test retrieval quality with representative questions from actual users.

2 Use hybrid retrieval

Combine vector similarity with keyword search and metadata filters for more reliable results.

3 Always surface citations

Show which source chunks informed an answer — builds trust and enables verification.

4 Instrument for groundedness

Log retrieved context alongside generated answers so you can audit and evaluate later.

5 Version your prompts

Treat prompt templates like code — track changes, test before deploying updates.

6 Design fallback paths

Plan what happens when retrieval returns nothing relevant, or a model call fails.

Common Issues & Pitfalls



Over- or under-chunking

Kills retrieval precision in both directions — this is the #1 quality issue in RAG systems.



Ignoring metadata filters

Treating every document as equally relevant when access rights, dates, or regions should narrow results.



No evaluation loop

Shipping without a way to measure relevance, groundedness, or hallucination rate over time.



Prompt injection via retrieved content

Malicious or malformed text inside a document can hijack the model's instructions.



Stale or unsynced index

Source documents change but the index never gets re-run, so retrieval quietly serves outdated answers.

Real-World Enterprise Scenario

CASE STUDY

A mid-size financial services firm needs employees to get instant, accurate answers to internal policy and compliance questions — currently a bottleneck routed entirely through a small compliance team.

Challenge

Policy documents update frequently; wrong answers carry real regulatory risk; the compliance team can't scale 1:1 with question volume.

RAG approach

All policy documents indexed in Azure AI Search with metadata (region, effective date, department); answers generated with mandatory citations back to source clauses.

Fallback for edge cases

When retrieval confidence is low or a question touches high-stakes topics, the system flags it for a human compliance reviewer instead of answering automatically.

Outcome

Most questions answered instantly with a verifiable citation; the compliance team now focuses only on genuinely ambiguous cases.

Applied Exercise: Build a Basic RAG Application

HANDS-ON ACTIVITY

Business Scenario

Northwind Traders, a mid-size retailer, has hundreds of internal FAQ and policy documents scattered across PDFs and wikis. Employees waste time hunting for answers or repeat the same questions in a support channel.

- 1 Employees can't quickly find answers buried in scattered documents
- 2 Keyword search misses answers that are phrased differently
- 3 The support/HR team keeps fielding the same repeat questions
- 4 Leadership wants a self-serve assistant employees can trust

Learning Objectives

- ✓ Load and chunk a real set of enterprise documents
- ✓ Generate embeddings and index them in Azure AI Search
- ✓ Build a retrieve → augment → generate query flow
- ✓ Display an answer together with its source references

Exercise Prerequisites & Azure Resources

1

Azure OpenAI Resource

A deployed chat model (e.g. gpt-4o) and a deployed embedding model (e.g. text-embedding-3-large).

2

Azure AI Search Resource

A search service on Basic tier or higher, with permission to create a new index.

3

Python Environment

Python 3.10+, with the openai and azure-search-documents packages installed.

4

Sample Document Set

The provided sample enterprise documents (FAQ, policy, and HR files) used throughout the exercise.

Sample Documents & Questions

EXAMPLES

1 remote-work-policy.pdf

Remote/hybrid eligibility, hours, and equipment. Sample Q: "How many days a week can I work from home?"

2 expense-reimbursement.pdf

Limits for submitting and approving expense claims. Sample Q: "What's the per-night hotel limit for travel?"

3 onboarding-faq.md

New-hire questions on benefits, IT setup, and first-week logistics. Sample Q: "When does my health insurance start?"

4 pto-and-leave-policy.pdf

Accrual and carryover rules, how to request time off. Sample Q: "How many PTO days carry over next year?"

Step-by-Step: Document Ingestion & Indexing

HANDS-ON ACTIVITY

```
# 1. Load and read the sample documents
docs = load_documents("./sample_docs")
print(f"Loaded {len(docs)} documents")

# 2. Split each document into chunks
chunks = []
for doc in docs:
    chunks += chunk_text(doc, size=400, overlap=50)
print(f"Created {len(chunks)} chunks")

# 3. Generate an embedding for each chunk (Azure OpenAI)
for chunk in chunks:
    chunk["vector"] = embed(chunk["text"])
    chunk["source_file"] = chunk["doc_name"]

# 4. Upload chunks + vectors into the Azure AI Search index
search_client.upload_documents(documents=chunks)
print("Indexing complete")
```

Steps & Tips

- Keep chunk size consistent with what you'll use at query time
- Store the source filename with each chunk — you'll need it to cite sources later
- Expected output: your Azure AI Search index shows a document count greater than zero
- Re-run ingestion any time a source document changes

Step-by-Step: Query & Grounded Response

HANDS-ON ACTIVITY

```
# 5. Accept the user's question
question = input("Ask a question: ")
# 6. Embed the question (same model as ingestion)
query_vector = embed(question)
# 7. Retrieve the top-K relevant chunks
results = search_client.search(
    search_text=question,
    vector_queries=[{
        "vector": query_vector,
        "k_nearest_neighbors": 5,
        "fields": "content_vector"
    }]
)
# 8. Construct the augmented prompt
context = "\n\n".join(r["content"] for r in results)
prompt = f"""Answer using ONLY the context below.
Context: {context}
Question: {question}"""
# 9. Generate the grounded answer and show sources
answer = chat_client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": prompt}]
)
print(answer.choices[0].message.content)
print("Sources:", [r["source_file"] for r in results])
```

Steps & Tips

- Same embedding model as ingestion — non-negotiable
- "Answer using ONLY the context" keeps the model from guessing
- Print the source filenames alongside the answer — this is your citation
- Try a question with no good match in your documents — check what the model does

Expected Output & Validation Steps

VALIDATION

- 1. Run a known-answer question** Ask a sample question you know the answer to; confirm the response matches the source document.
- 2. Check the citations** Open the returned source file and confirm the information used in the answer is actually in it.
- 3. Edit a document and re-index** Change a fact in one sample document, re-run ingestion, and ask the same question again.
- 4. Ask an out-of-scope question** Ask something not covered by any document; the system should say it doesn't know, not guess.

Success Criteria & Troubleshooting Tips

CHECKLIST

Success Criteria

- Answer is grounded in the sample documents, not generic knowledge
- Answer changes appropriately when you edit or add a document
- You can point to which retrieved chunk(s) informed the answer
- You can explain, in your own words, each of the seven pipeline stages

Troubleshooting Tips

- Generic answer → check the prompt actually includes the retrieved context
- No results returned → confirm you're querying the same field you embedded into
- Irrelevant chunks retrieved → chunk size is probably too large or too small
- Index looks empty → confirm the upload step succeeded and field names match your schema

Session Summary

- 1 RAG grounds LLMs — in your enterprise data, reducing hallucination without retraining.
- 2 The pipeline has 7 stages — ingest, chunk, embed, store, retrieve, augment, generate — four offline, three online.
- 3 Azure maps cleanly to it — Azure OpenAI (models), Azure AI Search (vector store), Document Intelligence (extraction).
- 4 Retrieval quality drives answer quality — chunking and embedding choices matter more than model choice alone.
- 5 You built it — a working basic RAG application, end to end, in the Applied Exercise.

Knowledge Check

DISCUSSION

Q1 Why can't you just ask an LLM about your company's internal policies directly?

Because the model's knowledge is frozen at training time and it has no access to your internal systems — it needs retrieved context to answer accurately.

Q2 What's the risk of making chunks too large?

Irrelevant text dilutes the retrieved passage's relevance and wastes context window space.

Q3 What's the key difference between RAG and fine-tuning?

RAG changes what the model sees at query time by retrieving context; fine-tuning changes the model's weights through retraining.

Q4 Name one Azure service and its role in the RAG pipeline.

E.g., Azure AI Search stores and retrieves vectors; Azure OpenAI generates embeddings and answers.

Key Takeaways

Retrieval quality (chunking + embeddings) determines answer quality far more than model choice alone.

Next step: point this same pattern at a real, small document set from your own team — that's the fastest way to find out where your chunking or retrieval needs tuning.